

CH4 - Clip 14.

서버 통신 이론



01

HTTP 개요

02

클라이언트와 서버

03

REST API 소개

04

MLOps에서의 활용

1. HTTP 개요

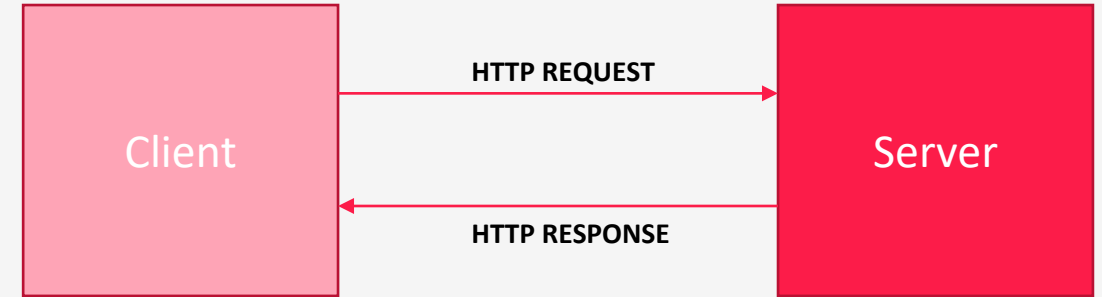
HTTP 란?

HTTP란 무엇인가?

- HTTP(HyperText Transfer Protocol)는 웹 상에서 데이터를 교환하기 위한 프로토콜
- 텍스트, 이미지, 비디오 등 다양한 형태의 데이터를 클라이언트와 서버 간에 전송
- 월드 와이드 웹의 기초적인 기술 중 하나로, 인터넷의 급속한 성장과 발전에 기여

요청 응답 프로세스

- 클라이언트(웹 브라우저 혹은 소비자)가 서버에 특정 리소스를 요청
- 서버는 이 요청을 처리하고 적절한 응답을 클라이언트에 반환



HTTP 란?

HTTP Method

- GET : 서버로부터 정보를 조회하기 위해 사용
- POST : 서버에 데이터를 전송하기 위해 사용
- PUT : 서버에 데이터를 업데이트할 때 사용
- DELETE : 서버의 리소스를 삭제할 때 사용
- 각 method는 특정한 유형의 액션을 정의하며, RESTful API 설계의 핵심 구성 요소

HTTP 상태코드

- 서버가 클라이언트의 요청을 어떻게 처리했는지 설명하는 코드
- 클라이언트는 이 코드를 통해 요청의 성공, 실패 여부를 알 수 있음
- 200 OK : 요청이 성공적으로 처리
- 404 Not Found : 요청한 리소스를 찾을 수 없음
- 500 Internal Server Error : 서버 내부 오류

2. 클라이언트와 서버

클라이언트와 서버

클라이언트 역할

- 웹 브라우저 : 사용자가 웹 페이지를 요청할 때, 웹 브라우저는 클라이언트의 역할 수행, HTTP를 통해 서버에 페이지 요청을 보내고, 받은 응답을 사용자에게 보여줌
- 응용 프로그램 : 이메일 클라이언트, 온라인 게임, 모바일 앱 등도 클라이언트의 예. 각 각의 애플리케이션은 서버와 통신하여 특정 기능을 수행

서버의 역할

- 데이터 처리 및 관리 : 서버는 클라이언트 요청을 받아 데이터를 처리하고 관리, 데이터베이스 서버, 파일 서버 등 다양한 유형 서버가 존재
- 자원 제공 : 웹 서버는 웹 페이지, 이미지, 비디오 등의 자원을 제공. 서버는 이러한 자원을 저장하고, 클라이언트 요청에 따라 전송됨

클라이언트와 서버 예시

웹 페이지 요청

- 사용자가 브라우저에서 URL을 입력하면, 브라우저(클라이언트)는 해당 주소의 서버에 웹 페이지를 요청
- 서버는 요청을 받고, 해당 웹 페이지의 HTML, CSS, Javascript 파일 등을 클라이언트에게 전송

이메일 전송

- 사용자가 이메일 클라이언트를 사용해 메일을 보낼 때, 이메일 서버는 메일을 받아 수신자의 서버로 전송

3. REST API 소개

REST API 소개

REST API란?

- REST API는 웹 표준을 기반으로 서버와 클라이언트 간의 통신을 구현하기 위한 인터페이스
- 이는 자원(Resource)의 표현에 의한 상태 전달을 의미하며, 웹의 기본 프로토콜인 HTTP를 사용

작동 원리

- REST API는 '자원(URI), 행위(HTTP Method), 표현(Representation)'의 세가지 구성 요소로 이루어짐
- 클라이언트는 URI를 통해 자원을 지정하고, HTTP 메서드를 통해 해당 자원에 대한 행위(생성, 조회, 수정, 삭제)를 지정

상태 비저장성(Statelessness)

- REST는 상태 비저장성을 가짐. 즉, 서버는 클라이언트 상태를 유지하지 않으며, 각 요청은 독립적으로 처리
- 서버 처리가 단순화되고 확장성이 증가

REST API 소개

REST API METHOD

- GET : 서버로부터 정보를 조회하기 위해 사용
- POST : 서버에 데이터를 전송하기 위해 사용
- PUT : 서버에 데이터를 업데이트할 때 사용
- DELETE : 서버의 리소스를 삭제할 때 사용
- 각 method는 특정한 유형의 액션을 정의

REST API 예시

온라인 서점 REST API 예시

- 온라인 서점을 위한 REST API 설계 진행.
- 책 정보를 조회하고 관리하기 위한 API로 기본적인 CRUD(Create, Read, Update, Delete) 작업을 수행

	목적	Endpoint	응답 예시
READ	특정 책의 세부 정보를 조회	GET /books/{bookId}	{ "bookId": "123", "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "year": 1925, "genre": "Fiction" }
CREATE	새로운 책을 목록에 추가	POST /books	{ "title": "1984", "author": "George Orwell", "year": 1949, "genre": "Dystopian Fiction" }
UPDATE	기존 책 정보 업데이트	PUT /books/{bookId}	{ "title": "1984", "author": "George Orwell", "year": 1949, "genre": "Political Fiction" }
DELTE	특정 책을 목록에서 삭제	DELETE /books/{bookId}	

4. MLOps에서의 HTTP & REST API 활용

MLOps에 적용

데이터 수집 및 처리

- 고객의 상호작용 데이터는 REST API를 통해 실시간으로 수집되며, 이 데이터는 머신러닝 모델 학습에 사용

모델 서빙

- 훈련된 모델은 REST API를 통해 제공되며, 웹사이트 및 모바일 앱에서 이 API를 호출하여 실시간 추천을 제공

모델 파이프라인 동작

- 트레이닝 파이프라인은 자동화되어 있으며, 새로운 데이터가 들어올 때 마다 모델을 재학습

MLOps 예시

대형 소매 업체의 상품 추천 시스템

- 문제 상황 : 다양한 상품을 취급하는 소매 업체가 온라인 고객에게 맞춤형 상품 추천을 제공
- 해결 방법 : 머신러닝 기반의 추천 시스템 개발. 모델을 고객의 구매 이력, 검색 행동, 상품 평가 데이터를 학습하여 개인화된 추천을 제공
- 구현 : HTTP와 REST API를 이용해 데이터 수집, 모델 서빙 을 관리하며, 주기적으로 모델 트레이닝을 하여 개인화된 추천의 성능을 향상